Project Report - Task 2

Data Storage Paradigms, IV1351

2024/11

Group 77 Project members:
[Sam Serbouti, serbouti@kth.se]
[Danni Noren, dannin@kth.se]

1 Introduction

This report details the work of building a hybrid version[1] of a logical and physical model derived from our conceptual model, which includes some minor changes from the original design. In the new model see figure 1, the monthlyRentFee is now handled in the Instrument entity. This approach eliminates redundancy, as the rental fee of each instrument is stored in the Instrument entity. Moreover, handling price changes is more efficient, as updates can be made directly in the Instrument entity.

Corrections for seminar 5

The price attribute was added to the pricingScheme entity to store the actual price for each lesson. Furthermore, to establish a relationship between lesson and instructor, instructorID attribute was added to lesson entity, allowing us to provide which instructor teaches a particular lesson.

The conceptual model, as well as the logical and physical model, have been updated in this document. Our GitHub repository, which includes the `Astah` files and SQL scripts, is available in the results section.

See Figure 1 to see the final conceptual model. Note that for the Instructor-Person inheritance, the symbol should be inverted—Instructor is a Person.

[1] In this model, the database tables are identified, the model is normalized, verifying all requirements will be met, column types and primary keys are used.

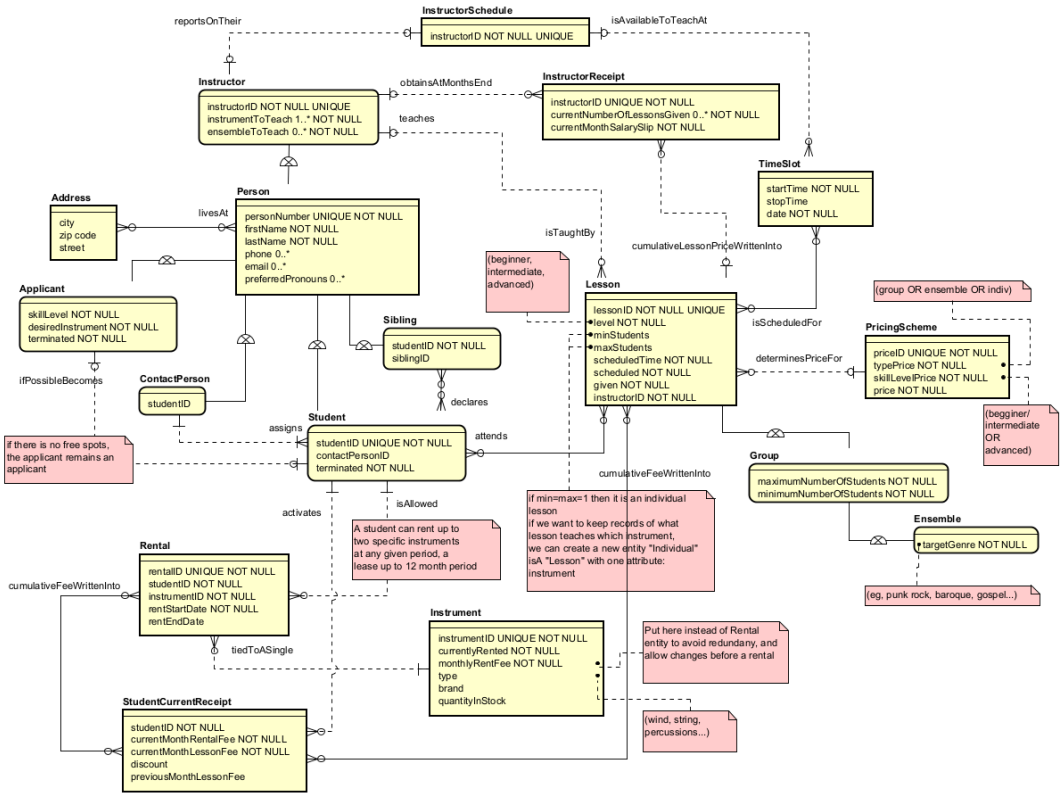

Figure 1: Conceptual Model

From the logical-physical model that will be describes here, we will build a database in the future hope to create an OLAP[2] which will not be covered here.

2 Literature Study

To complete this task, we have read chapters 5 and 8 of the 7th edition of *Fundamentals of Database Systems* by Ramez Elmasri and Shamkant Navathe. They respectively dealt with:

- relational data models : schemas, operations and their constrains
- relational algebra and relational calculus

We also followed a small online course on the topic of creating a logical model by Nicolas Rangeon on the online learning platform openclassrooms.

A page on Database.guide helped us understanding the basics of writing the SQL scripts for building the database.

[2]OLAP:Online Analytical Processing

This was in addition to the course's modules on relational models, SQL, logical and physical models and normalisation.

3 Method

To build the logical-physical model we used the Astah Professional diagram editor and their guide on data types and followed the following steps:

- **Create Tables for Entities**: For each entity in the conceptual model, create a corresponding table.
- **Add Columns for 0-1 or 1-1 Cardinality Attributes**: Add columns for attributes with at most one value (cardinality 0..1 or 1..1). If cardinality = 0..* for example: don't add it in the columns.
- **Create New Tables for Higher Cardinality Attributes**: Attributes with higher cardinality (ex: multiple emails or phone numbers) require new tables.
- **Specify Column Types**: for a time slot, with used a `TIMESTAMP` type. `BIT` was used for boolean variables and `DECIMAL` for float-type variables
- **Consider column constraints**: NOT NULL or UNIQUE[3]
- **Assigning Primary Keys to Strong Entities**: ie entities that have an id and a meaning on their own, that can exist independently of other entities (no total participation in relationships). We found the strong entities to be: student, person, instructor, applicant, rental, lesson, instrument and pricing_scheme.
- **Handling One-to-One and One-to-Many Relations**

We wrote SQL scripts manually with the help of GeeksForGeeks's guidelines on PostgreSQL. To populate the database, we used ChatGpt4 to generate random data since it proved to be the fastest and most flexible data generator.

4 Result

4.1 Logical-Physical model

The model is available in our github repository where you can directly download our `Astah` file (please note that the second version is the latest version of the `Astah` files including the corrections for seminar 5). But we added screenshots of this model in our report. See figure 2 to figure 7.

Note that for the table `rental`, we added as notes: `rent_start_date : "default value=TODAY" rent_end_date : "default value=TODAY+12 months constraint: rent_end_date>rent_start_date"` and the global note `"A student can rent up to two specific instruments at any given period, a lease up to 12 month period"`

[3] the uniqueness property of a column was indicated as a note on `Astah`

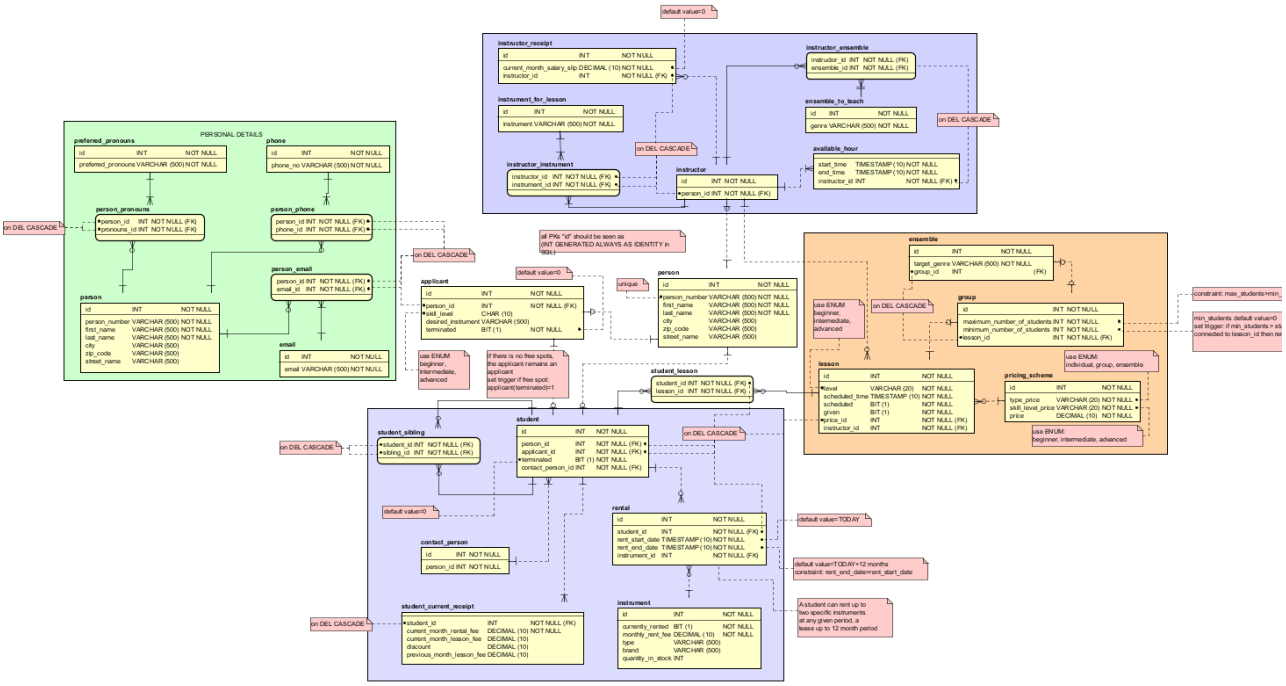

Figure 2: Screenshot of the complete logical-physical model in Astah Professional

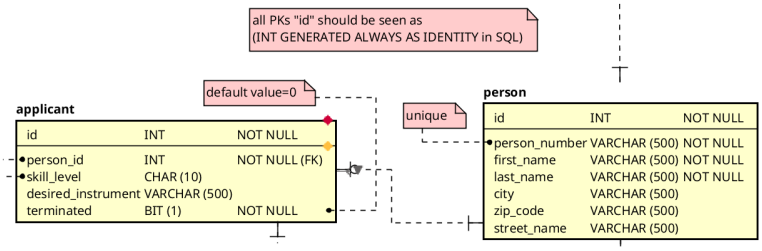

Figure 3: Screenshot of the logical-physical model in Astah focused on the table for `person`

4.2 Notes on building the logical-physical model

When we started working on the column types, we noticed how we could have new tables for instruments: one table called `instrument` for the instrument as a physical object that will be rented and one table called `instrument_for_lesson` for instruments as a concept: indeed, instructors must tell the staff which instruments they can teach, and applicants must tell the staff which instruments (there can be several) they want

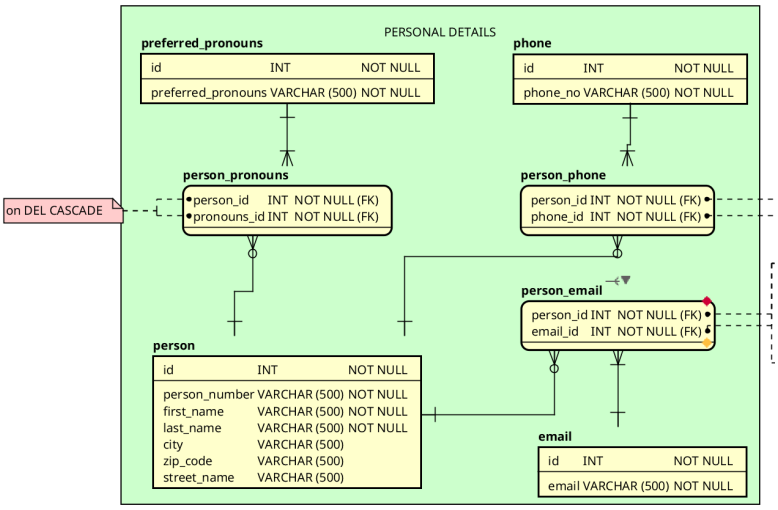

Figure 4: Screenshot of the logical-physical model in Astah focused on the different multivalued attributes of a person

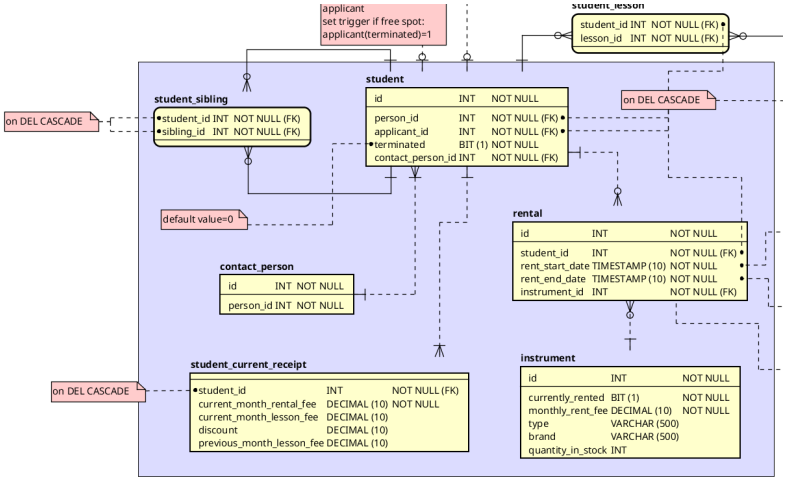

Figure 5: Screenshot of the logical-physical model in Astah focused on the table for `student`

to learn. To avoid dealing with long-text description typed columns, we decided to use tables with N:M relationships for these different concepts.

Additionally, we realised that our schedule entity meant for instructors to tell the staff when they are available, could be removed, and instead go directly to time slots linked to the instructor_id. The use of a time slot entity for the lessons was lost as soon as we noticed how `Astah` already had a `TIMESTAMP` data type.

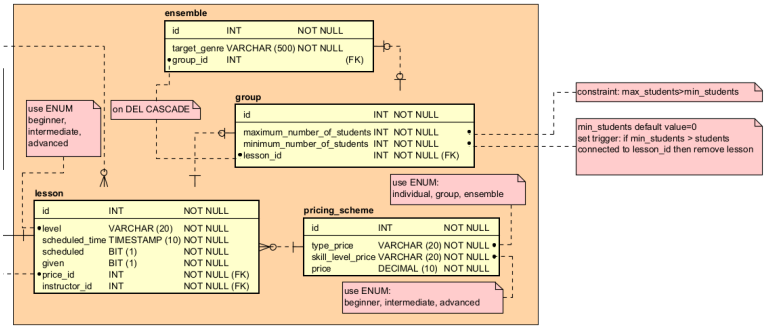

Figure 6: Screenshot of the logical-physical model in Astah focused on the table for `lesson`

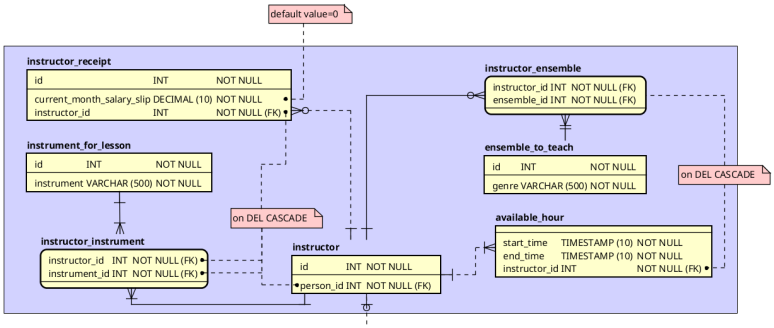

Figure 7: Screenshot of the logical-physical model in Astah focused on the table for `instructor`

4.3 SQL scripts

They can be found in the following github repository. We started our script for building the database by defining data types specific to our scenario:

```
-- Switch to postgres database
\c postgres;
-- Drop and create database
DROP DATABASE IF EXISTS soundgoodSEM2;
CREATE DATABASE soundgoodSEM2;
\c soundgoodSEM2;

-- ENUM TYPES
CREATE TYPE LEVEL AS ENUM ('beginner', 'intermediate', 'advanced');
CREATE TYPE INSTRUTYPE AS ENUM ('string', 'woodwind', 'brass', 'percussion',
CREATE TYPE LESSONTYPE AS ENUM ('individual', 'group', 'ensemble');
```

5 Discussion

When assigning PKs to strong entities, we chose not to use the person number as a PK but rather a surrogate key in order to avoid mixing real-world identifier with virtual ones. We figured the cost on disk space was minimal.

We moved the address fields (with zip, street, city) directly into the person table to avoid needing foreign keys and allow easier maintenance, despite slight denormalization.

We could have added a column in our `contact_person` table about the relationship between the student and the contact person (parent, older sibling etc) but figured that, since it wasn't in the customer requirements, we did not have to include it.

We tried to pay as much attention as possible on triggers, default values and domain constraints for our tables but some might have been left out. We considered them to be added details to the database (if a default value is missing, that will not impact too much the use of the database for the end-user).

We took the decision of propagating deletions via `CASCADE` for most of our columns and tables because we thought a *missing* value is always better than a *wrong* value.

We chose a data type `BIT(1)` instead of building a new `BOOLEAN` data type on Astah for simplicity and disk space. However, a single bit can be risky (in case of failure). This might have to be re-addressed.

Currently, our database provides a sort of archiving method : the customer can access old students who are marked as `terminated` the same way that they can access the current students records. This involves a cost in memory space but avoids using too much non-volatile storage for archiving. A discussion with the costumer needs to be held for this point.

We tried to make our logical-physical model as clear and concise as possible: no spider in the web, separated areas corresponding to real-life concepts and colours.

We didn't use `SERIAL` data type for primary keys, instead just marking them as `INT` for simplicity's sake. In our SQL script to build the database, we always marked them as: `id SERIAL PRIMARY KEY`.

We used quite a lot of cross-reference tables (especially in our PERSONAL DETAILS section). This was to keep 3NF but it can easily be taken out and put as single columns in the table `person` (using ENUM for pronouns and sticking to single inputs for phone, email and pronoun instead of several): it depends on the customer's choice for the balance between simplicity and normalization.

5.1 Self assessment

Criteria	**Notes**
Are naming conventions followed? Are all names sufficiently explaining?	The naming conventions are correctly followed, with all names written in lowercase and words separated by underscores, such as instrument_for_lesson.

Criteria	**Notes and discussion**
Is the crow foot notation correctly followed?	We used the IE notation for our diagram and payed attention to cardinality and the directions of the relationships.
Is the model in 3NF? If not, is there a good reason why not?	Most of the model is in 3NF. For example in the Conceptual model, the `address` was represented as separate entity. However in the logical and physical model, its attributes were moved to the person entity to simplify the relationships and reduce the need for additional foreign keys. We took out every multivalued attributes (only keeping atomic ones) by splitting them into atoms (for `address`) and splitting into tables. To reach 3NF, we mostly used cross reference tables.[4]
Are all tables relevant? Is some table missing?	All tables included are essential to meet the data requirements of the Soundgood music school. We did not delete any from our conceptual model that fit the customer requirements.
Are there columns for all data that shall be stored? Are all relevant column constraints and foreign key constraints specified? Can all column types be motivated?	Yes, the model includes columns for all data required from Soundgood school.All essential column constraints and foreign key constraints have been specified. Moreover all columns have the correct type corresponding with the data stored in them
Can the choice of primary keys be motivated? Are primary keys unique?	Yes the primary keys and composite primary keys are used to uniquely identify each table in the model.
Are all relations relevant? Is some relation missing? Is the cardinality correct?	Yes the relations were defined to meet the requirements from Soundgood school. Corssrefrence tables, sush as `student_lesson` were used to handle the many-to-many relationships. These tables combine the primary keys to create composite primary keys from the related tabales.

[4] As a consequence, we have a lot of foreign keys, but this allows decomposition.

Criteria	**Notes and discussion**
Is it possible to perform all tasks listed in the project description?	Yes the logical and physical model was designed to compute all requirements of Soundgood school available to us however to ensure all requirements are met the Conceptual model need to be analyzed with the school.
Are all business rules and constraints that are not visible in the diagram explained in plain text?	For the business rule on renting up to 2 specific instruments with a lease of up to 12 month period, we added a note on this in the table `rental`. We also proposed default values on the rent dates that fit this rule. On the rule that an applicant can become a student if there is free place, we added a note between the two tables.
Are there attributes which are calculated from other attributes and then written back to the database (derived attributes)? If so, why? Records of student fees and instructor payments might be examples of such attributes	Yes, some do. We believe we took a safe approach when building our model: some columns[5] can be derived from others especially when it comes to receipts and salary slips. But this way of thinking allows human-verifications when it comes to invoices and receipts. Furthermore, our assignment did not require us to focus on financial accounting.
Are tables (or ENUMs) always used instead of free text for constants such as the skill levels (beginner, intermediate and advanced)?	We used `ENUM` for a student's level, for the types of instrument (wood, brass, string etc) and the type of lessons (individual, group, ensemble). We could have added one for the target genre of an ensemble but realised that it would have been very long so we opted out for free text. However, with a discussion with the customer on this point, a list of target genres can quickly be written into our `DBbuild.sql` script.
Is the method and result explained in the report? Is there a discussion? Is the discussion relevant ?	We tried to address the most important aspects of our choices in the Discussion.

[5] like current_month_rental_fee and lesson fee for students and current_month_salary_slip for instructor that are cumulated from a lesson's type price

Criteria	Notes and discussion

Table 1: Self-assessment based on the task's criteria

6 Comments About the Course

The lecture on normalization was very useful to our work. A bit more preparation on SQL scripts could have been good as well but it was an opportunity for us to seek knowledge outside of the course's framework.